.Web Technologies/2026-2027/Lab 4

Teaching Staff

- Ștefănigă Sebastian-Aurelian
- Morariu Ioana-Alexandra
- Munteanu Alexandru

Objectives

- Familiarization with the **JavaScript language**, exploring syntax, events, and DOM manipulation.
- Applying **validations** on the login form to verify the correctness of the email address and password, providing relevant error messages.
- Creating an interactive **chatbot page** where the user can answer questions and receive feedback.

Instructions

Step 0. Open the project from the previous session and add new pages for the project.

```
project-lab/
|
|-- HTML/
|   |-- loading.html
|   |-- login.html
|   |-- register.html
|   |-- home.html
|   |-- chatbot.html
|
|-- CSS/
|   |-- style.css
|   |-- loading_style.css
|   |-- login_style.css
|   |-- register_style.css
|   |-- home_style.css
|   |-- chatbot_style.css
|
```

```
|-- JS/
|   |-- chatbot_js.js
|   |-- login_js.js
...
```

Step 1. Ensure that the login form is correctly created and functions properly.

```
<form id="loginForm">
      <input type="email" id="email" placeholder="Email">
      <input type="password" id="password" placeholder="Parola">
      <button type="submit">Login</button>
</form>
```

Step 2. Create a JavaScript script to validate the login form, ensuring that the email follows the UVT format (first.last[optional 2 digits]@e-uvt.ro) and that the password contains at least 7 characters, including uppercase and lowercase letters, digits, and a special character.

```
let loginForm = document.getElementById("...");
let emailInput = document.getElementById("...");
let passwordInput = document.getElementById("...");
loginForm.addEventListener("submit", function(e) {
  e.preventDefault();

  const email = emailInput.value.trim();
  const password = passwordInput.value;
  const emailRegex = ...
  const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{7,}$/;

  if (!emailRegex.test(email)) {
    alert(...);
    return;
  }

 if (!passwordRegex.test(password)) {
    alert(...);
    return;
  }
```

```
  alert("Login reușit!");
   window.location.href = "...";
});
```

Step 3. Integrate your JavaScript script into the HTML file to enable the validation functionality

```
   <script src="..."></script>
```

Step 4. Next, we will create the HTML skeletons for the chatbot

```
<!DOCTYPE html>
<html lang="en">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Chatbot</title>
<link rel="stylesheet" href="...">
</head>

<body>
<div class="chat-container" id="chat-container"></div>
<form id="chat-form">
<input type="..." id="..." placeholder="...">
<button type="submit">Trimite</button>
</form>
<script src="..."></script>
</body>
</html>
```

Step 5. Let's add the minimal CSS required for basic styling

```
body {
    background-color: …;
    display: flex;
    justify-content: …;
    align-items: center;
    height: …;
    margin: 0;
```

```
    flex-direction: column;
}

.chat-container {
    max-width: ...;
    width: ...;
    height: ...;
    padding: ...;
    background-color: ...;
    border-radius: ...;
    box-shadow: ...;
    margin-bottom: ...;
    overflow-y: ...;
}

.message {
    margin-bottom: ...;
}

#chat-form {
    display: flex;
}
```

The provided code contains the basic structure, but you are free to add additional CSS rules or formatting to tags, classes, and IDs to enhance the page's appearance.

Step 6. Add the JS code

```
let questions = [
  {
    question: "Te-ai apucat de ideea de proiect?",
    options: {
      a: "Da, am deja o schiță!",
      b: "Nu încă, dar mă gândesc la ceva",
      c: "Nu prea știu ce să fac..."
    },
    correctAnswer: "a",
    correctResponse: "Excelent! Abia aștept să văd ce iese!",
    incorrectResponse: "Nu-i nimic! Discutăm idei împreună"
  },

  {
    question: "Cum merge echipa ta la proiect?",
    options: {
```

```
      a: "Ne organizăm bine!",
      b: "Mai greu cu sincronizarea...",
      c: "Sunt singur"
    },
    correctAnswer: "a",
    correctResponse: "Foarte tare! Munca în echipă e cheia succesului!",
    incorrectResponse: "O să găsim soluții împreună, nu-ți face griji"
  }
  ...
];


let currentQuestionIndex = ...;
let chatContainer = document.getElementById("...");
let chatForm = document.getElementById("...");
let ...= document.getElementById("user-input");

displayQuestion();

function displayQuestion() {
  let currentQuestion = questions[...];
  let optionsHTML = Object.keys(currentQuestion.options)
    .map(key => `${key}. ${currentQuestion.options[key]}`)
    .join(" | ");

  let botResponse = document.createElement("div");
  botResponse.classList.add("message");
  botResponse.innerHTML = `<strong>Bot:</strong>
${currentQuestion.question}<br>${optionsHTML}`;

  chatContainer.appendChild(...);
  scrollChatContainerToBottom();
}

function scrollChatContainerToBottom() {
  chatContainer.scrollTop = chatContainer.scrollHeight;
}

chatForm.addEventListener("submit", function(event) {
  event.preventDefault();

  let userResponse = userInput.value.toLowerCase().trim();
  if (userResponse === "") return;
```

```
  let userMessage = document.createElement("div");
  userMessage.classList.add("message");
  userMessage.innerHTML = `<strong>You:</strong> ${userResponse}`;
  chatContainer.appendChild(...);


  let currentQuestion = questions[...];
  let botResponse = document.createElement("...");
  botResponse.classList.add("message");
  botResponse.innerHTML = `<strong>...</strong> `;

  if (... === currentQuestion.correctAnswer) {
    botResponse.... += currentQuestion.correctResponse;
  } else {
    botResponse.innerHTML += currentQuestio....;
  }

  chatContainer.appendChild(botResponse);

  currentQuestionIndex = (currentQuestionIndex + ...) % questions.length;
  userInput.value = "";

  setTimeout(() => {
    displayQuestion();
    scrollChatContainerToBottom();
  }, 800);
});
```

Step .7 Link the pages together to enable smooth navigation.

GOOD LUCK!